Practice Abstraction in Java

26 January 2026 19:47

Question 1: Abstract Class and Runtime Polymorphism

Create an abstract class **Employee** with the following specifications:

Abstract Class: Employee

- Data members:
  - int empId
  - String name
- Constructor to initialize data members.
- Abstract method:
  - double calculateSalary()
- Concrete method:
  - void showDetails() → displays employee details.

Subclasses:

1. **PermanentEmployee**
   - Additional data member: basicPay
   - Salary = basicPay + 40% HRA + 20% DA
2. **ContractEmployee**
   - Additional data member: hoursWorked, ratePerHour
   - Salary = hoursWorked × ratePerHour

Main Tasks:

- Store objects of both subclasses in an Employee array.
- Use a loop to display details and calculated salary.
- Demonstrate **runtime polymorphism.**

Question 2: Shape Drawing Using Abstract Class

Create an abstract class **Shape.**

Shape (Abstract Class)

- Abstract method: void draw()
- Concrete method: void info() → prints "This is a geometric shape"

Subclasses:

- Circle
- Rectangle
- Triangle

Each subclass should:

- Override draw() and print its own drawing message.

Tasks:

- Create an array of Shape references.
- Use a loop to call draw() and info().
- Explain how abstraction and polymorphism are achieved.

Question 3: Interface-Based Payment System

Create an interface **Payment.**

Interface: Payment

- Method: void payAmount(double amount)

Implementing Classes:

1. CreditCardPayment
2. DebitCardPayment
3. UPIPayment

Each class should:

- Print payment mode and amount paid.

Main Tasks:

- Store objects in an ArrayList<Payment>.
- Traverse using enhanced for-loop.
- Demonstrate **interface-based polymorphism.**

Question 4: Multiple Inheritance Using Interfaces

Create two interfaces:

Interface 1: Printable

- Method: void print()

Interface 2: Scannable

- Method: void scan()

Class:

- MultiFunctionDevice implements both interfaces.

Tasks:

- Implement both methods.
- Create object and invoke both methods.
- Explain how Java supports multiple inheritance using interfaces.

Question 5: Interface Inheritance Program

Create two interfaces:

Interface 1: Readable

- Method: void read()

Interface 2: Writable extends Readable

- Method: void write()

Class:

- Notebook implements Writable.

Tasks:

- Implement both methods.
- Create main method to demonstrate interface inheritance.
- Explain **IS-A relationship**.

Question 6: Default and Static Methods in Interface

Create an interface **Displayable** with:

- Static method: print(String msg)
- Default method: showInfo()

Class:

- DisplayClass implements Displayable

Tasks:

- Call static method using interface name.
- Call default method using object reference.
- Explain difference between static and default methods in interface.

Question 7: Factory Method with Interface

Create an interface **Logger**.

Logger

- Method: void log(String message)

Class:

- FileLogger implements Logger

Factory Class:

- LoggerFactory
  - Static method getLogger() returns Logger reference.

Tasks:

- Use factory method to create object.
- Call log() method.
- Explain purpose of factory design pattern.

Question 8: Abstract Class Implementing Interface

Create an interface **Printable** with method print().

Create an abstract class **AbstractPrinter** that:

- Implements Printable
- Does not implement print()

Create a concrete class **LaserPrinter** that:

- Extends AbstractPrinter
- Implements print()

Tasks:

- Demonstrate object creation and method call.
- Explain why abstract class is useful here.

Question 9: Nested Interface Program

Create an outer interface **Messageable**:

- Method: void msg()

Inside it, create a nested interface **PrintableMessage**:

- Methods:
  - void msg()
  - void printMsg()

Classes:

1. OuterMessage implements Messageable
2. InnerMessage implements Messageable.PrintableMessage

Tasks:

- Implement all methods.
- Create main class to demonstrate nested interface usage.
- Explain scope of nested interfaces.

Question 10: Abstract Class vs Interface Case Study

Design a program for a **Smart Device System.**

Requirements:

- Create an interface Connectable with method connect().
- Create abstract class SmartDevice with:
  - Abstract method operate()
  - Concrete method powerOn()

Classes:

- SmartTV extends SmartDevice and implements Connectable
- SmartWatch extends SmartDevice and implements Connectable

Tasks:

- Override all required methods.
- Store objects using abstract class reference.
- Demonstrate:
  - Abstraction
  - Interface implementation
  - Polymorphism